

PCAD
Programmazione Concorrente
Algoritmi Distribuiti

Arnaud Sangnier
arnaud.sangnier@unige.it

MUTUA ESCLUSIONE ANCORA

Problema della sezione critica

Algoritmo di Dekker

```
int turn=1; //variabili condivise  
bool D1=false;  
bool D2=false;
```

```
Process P1  
while(true){  
p1: SNC  
p2: D1=true;  
p3: while(D2==true){  
p4:  if(turn==2){  
p5:   D1=false;  
p6:   while(turn!=1){}  
p7:   D1=true;}}  
p8: SC  
p9: turn=2;  
p10:D1=false;}
```

```
Process P2  
while(true){  
q1: SNC  
q2: D2=true;  
q3: while(D1==true){  
q4:  if(turn==1){  
q5:   D2=false;  
q6:   while(turn!=2){}  
q7:   D2=true;}}  
q8: SC  
q9: turn=1;  
q10:D2=false;}
```

Quale proprietà verifica l'algoritmo ? Come verificarle?

Problema della sezione critica

Algoritmo di Dekker - Proprietà

- Un **invariante** è una proprietà che è vera in tutti gli stati
- Un **invariante induttivo** è una proprietà che è vera in uno stato e per **tutti** gli stati (anche quelli non raggiungibili), se è vera allora è vera in ogni stato successivo

Proprietà:

L'algoritmo di Dekker verifica gli invarianti (induttivi) successivi:

- $\text{turn} == 1 \text{ or } \text{turn} == 2$
- $(p3 \text{ or } p4 \text{ or } p5 \text{ or } p8 \text{ or } p9 \text{ or } p10) \iff D1 == \text{true}$
- $(q3 \text{ or } q4 \text{ or } q5 \text{ or } q8 \text{ or } q9 \text{ or } q10) \iff D2 == \text{true}$

Prova:

- il valore iniziale di turn è 1 e le uniche istruzioni che modificano turn sono o $\text{turn} = 1$ o $\text{turn} = 2$
- D1 è modificato solo da P1 e D2 solo da P2

Problema della sezione critica

Algoritmo di Dekker - Proprietà

Teorema:

L'algoritmo di Dekker verifica la mutua esclusione.

Prova:

- Supponiamo che P1 arriva in SC (quindi in p8)
- Allora $D2 == \text{false}$ (usciamo del while)
- Grazie al invariante
 $(q3 \text{ or } q4 \text{ or } q5 \text{ or } q8 \text{ or } q9) \iff D2 == \text{true}$
P2 non può essere anche lui in q8

```
Process P1
while(true){
p1: SNC
p2: D1=true;
p3: while(D2==true){
p4:  if(turn==2){
p5:   D1=false;
p6:   while(turn!=1){}
p7:   D1=true;}}
p8: SC
p9: turn=2;
p10:D1=false;}
```

Problema della sezione critica

Algoritmo di Dekker - Proprietà

Teorema:

L'algoritmo di Dekker verifica l'assenza di deadlock.

Prova:

- Supponiamo che c'è un deadlock.
- P1 e P2 sono entrambi 'bloccati' nel pre-protocollo
- Supponiamo che $turn == 1$ (NB: il pre-protocollo non cambia il valore di $turn$)
- Allora P1 finirà per essere in loop fra p3 e p4
- E P2 finirà per essere in loop su q6 (perché $turn == 1$)
- Quindi finiremmo per avere $D2 = false$ grazie a l'invariante ($q3 \text{ or } q4 \text{ or } q5 \text{ or } q8 \text{ or } q9$) $\Leftrightarrow D2 == true$
- **Ma allora P1 non può essere in loop fra p3 e p4**

```
Process P1
while(true){
p1: SNC
p2: D1=true;
p3: while(D2==true){
p4:  if(turn==2){
p5:   D1=false;
p6:   while(turn!=1){}
p7:   D1=true;}}
p8: SC
p9: turn=2;
p10:D1=false;}
```

Problema della sezione critica

Algoritmo di Dekker - Proprietà

Teorema:

L'algoritmo di Dekker verifica l'assenza di starvation sotto le seguenti ipotesi:

- equità fra i processi (se un processo può eseguire un'istruzione, la eseguirà prima o poi)
- la SC termina sempre e finisce con l'esecuzione del post-protocollo

```
int turn=1; //variabile condivisa  
bool D1=false;  
bool D2=false;
```

```
Process P1  
while(true){  
p1: SNC  
p2: D1=true;  
p3: while(D2==true){  
p4:  if(turn==2){  
p5:   D1=false;  
p6:   while(turn!=1){}  
p7:   D1=true;}}p8: SC  
p9: turn=2;  
p10:D1=false;}
```

```
Process P2  
while(true){  
q1: SNC  
q2: D2=true;  
q3: while(D1==true){  
q4:  if(turn==1){  
q5:   D2=false;  
q6:   while(turn!=2){}  
q7:   D2=true;}}q8: SC  
q9: turn=1;  
q10:D2=false;}
```

Problema della sezione critica

Algoritmo di Dekker - Prova dell'assenza di starvation

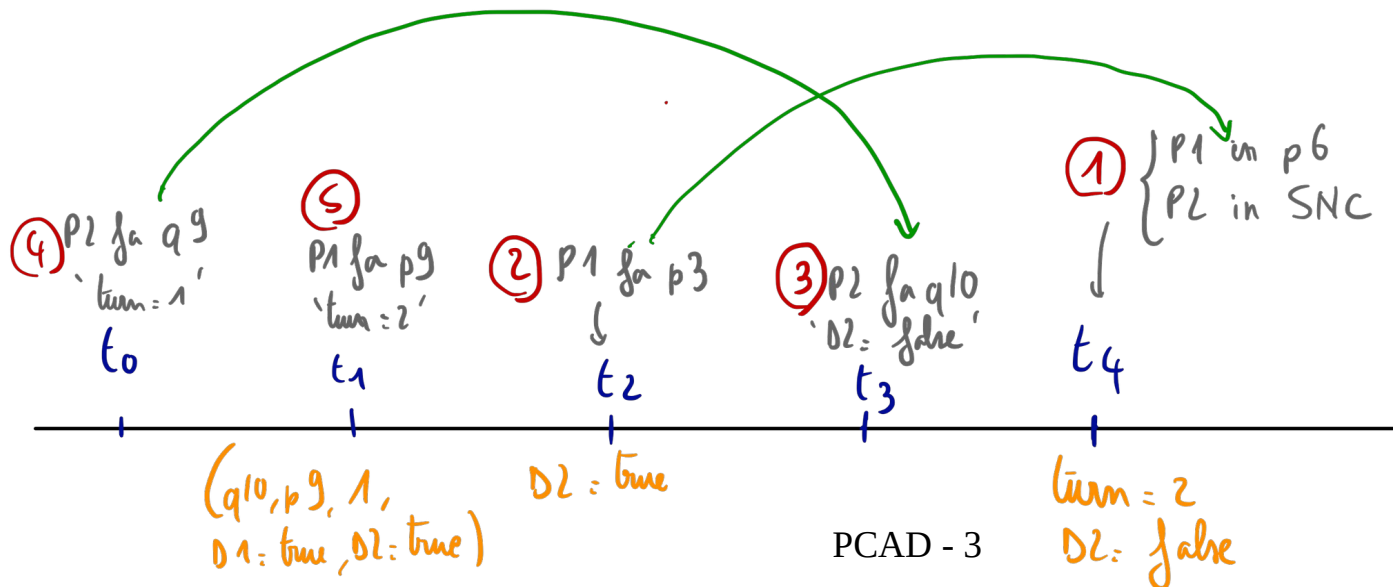
- Supponiamo che P1 sia in starvation. Dunque P1 è quindi bloccato nel suo pre-protocollo. Dove si trova P2 ?
 - Non può essere bloccato nel pre-protocollo (assenza di deadlock)
 - Se non è bloccato nella sua SNC e richiede la SC **infinitamente spesso**, allora dopo aver avuto accesso alla SC:
 - mette turn a 1 dando la priorità a P1
 - turn non cambierà più, quindi P1 non può essere bloccato in p6
 - allora P1 è bloccato fra P3 e P4 e P2 finirà bloccato in q6
 - perché $turn == 1$ e $D1 == true$
 - ma allora abbiamo un deadlock -> **contraddizione**

```
Process P1
while(true){
p1: SNC
p2: D1=true;
p3: while(D2==true){
p4:  if(turn==2){
p5:   D1=false;
p6:   while(turn!=1){}
p7:   D1=true;}}
p8: SC
p9: turn=2;
p10:D1=false;}
```

Problema della sezione critica

Algoritmo di Dekker - Prova dell'assenza di starvation

- Se P2 è bloccato nella sua SNC
 - **t4**: tempo in cui P2 è bloccato in SNC e P1 è bloccato nel pre-protocollo
 - **D2==false** per tutto il futuro, quindi P1 è bloccato in p6 e **turn==2**
 - **t2**: ultimo momento in cui P1 ha fatto p3 ($t_2 < t_4$)
 - come P1 è in p6 in t4, in t2 abbiamo **D2==true**
 - fra t2 e t4, D2 è cambiato, dunque in **t3**, P2 ha fatto q10 ($t_2 < t_3 < t_4$)
 - prima di t3, al tempo **t0**, P2 ha fatto q9 (mettendo **turn=1**) ($t_0 < t_3$) (e P2 non fa nulla fra t0 e t3)
 - dopo t0, in t1, P1 ha fatto p9 e abbiamo $t_1 < t_2$
 - in t1, abbiamo lo stato
 - **(p9, q10, turn=1, D1=true, D2=true)**



PCAD - 3

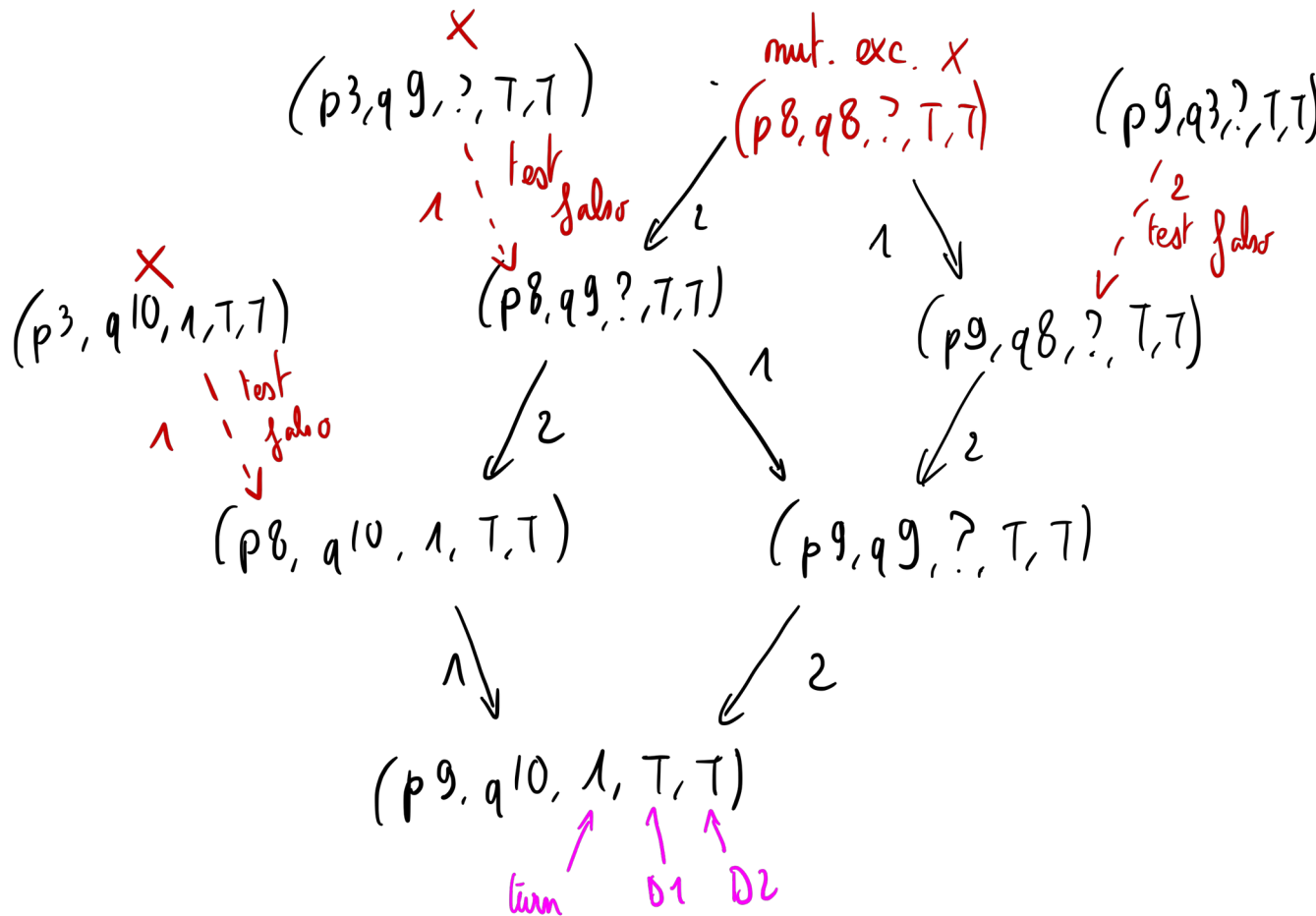
```

Process P1
while(true){
  p1: SNC
  p2: D1=true;
  p3: while(D2==true){
  p4:  if(turn==2){
  p5:   D1=false;
  p6:   while(turn!=1){}
  p7:   D1=true;}}
  p8: SC
  p9: turn=2;
  p10: D1=false;}
    
```

Problema della sezione critica

Algoritmo di Dekker - Prova dell'assenza di starvation

- Lo stato (**p9,q10,turn=1,D1=true, D2=true**) non è raggiungibile (prova con computazione 'in dietro')



```

Process P1
while(true){
  p1: SNIC
  p2: D1=true;
  p3: while(D2==true){
  p4:  if(turn==2){
  p5:   D1=false;
  p6:   while(turn!=1){}
  p7:   D1=true;}}
  p8: SC
  p9: turn=2;
  p10: D1=false;}
    
```

Altri algoritmi di mutua esclusione

Volume 12, number 3

INFORMATION PROCESSING LETTERS

13 June 1981

MYTHS ABOUT THE MUTUAL EXCLUSION PROBLEM

G.L. PETERSON

Department of Computer Science, University of Rochester, Rochester, NY 14627, U.S.A.

Received 13 January 1981; revised version received 30 March 1981

Parallelism, mutual exclusion

Recently in these pages appeared a report by Doran and Thomas [2] which gave partially simplified versions of Dekker-like solutions to the two process mutual exclusion problem with busy-waiting. This report presents a truly simple solution to the problem and attempts in a small way to dispel some myths that seem to have arisen concerning the problem.

Briefly, the *mutual exclusion problem* for two processes is to find sections of code (*trying protocol*, *exit protocol*) for each of two asynchronous processes to use when trying to enter and upon exiting their designated critical sections. The protocols must preserve mutual exclusion and not have deadlock or lockout. *Mutual exclusion* means that both processes can never be in their critical sections at the same time. No *deadlock* or *lockout* means that no process waits forever inside a protocol. More formal definitions can be found in [5] and elsewhere.

The original solution due to Dekker is discussed at length by Dijkstra in [1]. Of the many reformulations given since, perhaps the best appears in [3]. (Unfortunately the authors believe their correct solution is incorrect.) The solutions of Doran and Thomas are slight improvements which eliminate the 'loop inside a loop' structure of the previously published solutions.

The protocols of P_1 and P_2 are given in Fig. 1. Q_1 and Q_2 are initially *false* and $TURN$ may start as either 1 or 2. (The busy wait loop '*wait until Boolean*' is just another way of saying "*repeat/* empty statement/* until Boolean*". The Boolean formula is *not* evaluated atomically.)

As can be seen, the algorithm has a very simple structure. This results in an easy proof of correctness. First, neither process can be locked out. Consider P_1 , it has only one wait loop, and assume it can be forced to remain there forever. After a finite amount of time, P_2 will be doing one of three general things: not trying to enter, waiting in its protocol, or repeatedly cycling through its protocols. In the first case, P_1 notes that Q_2 is *false* and proceeds. The second case is impossible due to $TURN$ being either 1 or 2; and one of the processes will proceed. In the third case P_2 will quickly set $TURN$ to 2 and never change it back to 1, allowing P_1 to proceed.

If mutual exclusion were not preserved and both processes could somehow end up in their critical sections at the same time, then we have $Q_1 = Q_2 = \text{true}$. Their tests in their wait loops just prior to entering their critical sections at this point could not have been at approximately the same time as $TURN$ would have

Storia

- Algoritmo di Dekker: 1964
- Algoritmo di Peterson: 1981
 - Più facile di dimostrare la sua correttezza
 - Più facile da adattare per $N > 2$ processi

Problema della sezione critica

Algoritmo di Peterson

```
int turn=1; //variabili condivise  
bool D1=false;  
bool D2=false;
```

```
Process P1  
while(true){  
p1: SNC  
p2: D1=true;  
p3: turn=2;  
p4: while(turn!=1 &&D2==true){}  
p5: SC  
p6:D1=false;}
```

```
Process P2  
while(true){  
q1: SNC  
q2: D2=true;  
q3: turn=1;  
q4: while(turn!=2 &&D1==true){}  
q5: SC  
q6:D2=false;}
```